



DB

PIZZA SALES

SQL Portfolio — Questions & Answers

13 SQL queries · Filtering, Aggregation, Joins, Subqueries & Window Functions

Q1. Retrieve the total number of orders placed

```
SELECT COUNT(order_id)
FROM orders;
```

Q2. Calculate the total revenue generated from pizza sales

```
SELECT ROUND(SUM(order_details.quantity * pizzas.price), 2)
FROM order_details
JOIN pizzas ON order_details.pizza_id = pizzas.pizza_id;
```

Q3. Identify the highest-priced pizza

```
SELECT price FROM pizzas
ORDER BY price DESC
LIMIT 1;
```

Q4. Identify the most common pizza size ordered

```
SELECT pizzas.size, COUNT(order_details.order_details_id)
FROM pizzas
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizzas.size;
```

Q5. List the top 5 most ordered pizza types along with their quantity

```
SELECT pizza_types.name, COUNT(order_details.quantity) AS quantity
FROM pizza_types
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```

Q6. Join the necessary tables to find the total quantity of each pizza category ordered

```
SELECT pizza_types.category, SUM(order_details.quantity) AS quantity
FROM pizza_types
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.category
ORDER BY quantity DESC;
```

Q7. Determine the distribution of orders by hour of the day

```
SELECT COUNT(order_id), HOUR(order_time)
FROM orders
GROUP BY HOUR(order_time)
ORDER BY COUNT(order_id) DESC;
```

Q8. Join the necessary table to find the category-wise distribution of pizzas

```
SELECT category, COUNT(name)
FROM pizza_types
GROUP BY category;
```

Q9. Determine the top 3 most ordered pizza types based on revenue

```
SELECT pizza_types.name,
SUM(order_details.quantity * pizzas.price) AS revenue
FROM pizza_types
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Q10. Group orders by date and calculate the average number of pizzas ordered per day

```
SELECT ROUND(AVG(quantity), 2) FROM (  
SELECT orders.order_date, SUM(order_details.quantity) AS quantity  
FROM orders  
JOIN order_details ON orders.order_id = order_details.order_id  
GROUP BY orders.order_date  
) AS order_quantity;
```

Q11. Calculate the percentage contribution of each pizza type to total revenue

```
SELECT pizza_types.category,  
(SUM(order_details.quantity * pizzas.price) /  
(SELECT SUM(order_details.quantity * pizzas.price)  
FROM order_details  
JOIN pizzas ON order_details.pizza_id = pizzas.pizza_id)  
) * 100 AS revenue  
FROM pizza_types  
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id  
GROUP BY pizza_types.category;
```

Q12. Analyse the cumulative revenue generated over time

```
SELECT order_date, SUM(revenue) OVER (ORDER BY order_date) AS cum_rev  
FROM (  
SELECT orders.order_date,  
SUM(order_details.quantity * pizzas.price) AS revenue  
FROM orders  
JOIN order_details ON orders.order_id = order_details.order_id  
JOIN pizzas ON order_details.pizza_id = pizzas.pizza_id  
GROUP BY orders.order_date  
) AS sales_data;
```

Q13. Determine the top 3 most ordered pizza types based on revenue, for each pizza category

```
SELECT category, name, revenue FROM (
SELECT category, name, revenue,
RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
FROM (
SELECT pizza_types.category, pizza_types.name,
SUM(order_details.quantity * pizzas.price) AS revenue
FROM pizza_types
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.category, pizza_types.name
) AS a
) AS b
WHERE rn <= 3;
```